University of Kansas

Algorithmic Algebraic Parser

Test Case

Version <1.2>

Revision History

Date	Version	Description	Author
<mm/dd/yy>	<x.x>	<details>	<name>
04/29/26	1.0	Filling out initial details	Nathan Phillips
04/30/26	1.1	Adding additional test cases	Nathan Phillips
04/30/26	1.2	Adding additional test cases	Nathan Phillips

Table of Contents

1. Purpose 4

2. Test case identifier 4

3. Test item 4

4. Input specifications 4

5. Output specifications 4

6. Environmental needs 4
   6.1.1 Other 4

7. Special procedural requirements 5

8. Intercase dependencies 5

Test Case

1. Purpose

The Test Case Specification document for the Algorithmic Algebraic Parser defines a series of test cases for items that should be tested to verify the completeness, accuracy, and total functionality of the project. The Specification also includes necessary environment variables for reproducing test cases.

2. Test Case Table

ID	Name	Purpose	Inputs	Expected Outputs
TC01	Basic Verification	Verify basic addition functionality.	1+1	2
TC02	Unclosed Left Parenthesis	Test error against an unclosed left parenthesis.	((1+1)	Error message
TC03	Unmatched Right Parenthesis	Test system's ability to report unmatched right parentheses.	(1+1))	Error message
TC04	Division by Zero (Direct)	Verify system intercepts and clearly reports division by zero without crashing.	4/0	"Division by zero" error
TC05	Modulo by Zero	Verify system intercepts and clearly reports modulo by zero.	10%0	"Modulo by zero" error
TC06	Missing Operator	Test error handling when an operator is missing between operands (implicit multiplication is not explicitly supported).	5(2+3)	Error message
TC07	Missing Operand	Test error handling for consecutive binary operators missing an operand.	4+*2	Error message
TC08	Invalid Character	Ensure the system reports invalid alphabetical characters.	3+a	"Invalid character/operator" error
TC09	Empty Input	Verify handling of an empty input string when Enter is pressed.	\n	Error message
TC10	Right-to-Left Exponentiation	Test right-to-left associativity of the exponentiation operator.	2**3**2	512
TC11	Left-to-Right Subtraction	Test left-to-right associativity of standard operators.	10-4-2	4
TC12	Strict PEMDAS Enforcement	Verify standard order of operations mixing arithmetic and exponentiation.	2+3*4**2	50
TC13	Extraneous Whitespace	Verify the system correctly handles and ignores heavy extraneous whitespace between tokens.	4 * 2	8
TC14	Unary Operators	Verify correct application of multiple unary negation and addition operators.	-(+1)+(+2)	1
TC15	Extraneous Parentheses	Ensure extraneous nested parentheses are processed without affecting expression validity.	(((3+2)))	5
TC16	Empty Parentheses	Test error handling for parentheses lacking any internal operands or operators.	( )	Error message
TC17	Trailing Binary Operator	Test error reporting for missing operand at the end of the input string.	5+3-	Error message
TC18	Leading Binary Operator	Test error reporting for missing operand at the beginning of the input string.	/5+2	Error message
TC19	Sequential Unary and Binary	Verify the parser can distinguish a binary operator followed by a unary operator.	5+-3	2

TC20	Complex PEMDAS & Modulo	Test step-by-step calculation strictly following PEMDAS with mixed operators.	4*(3+2)%7-1	5
TC21	Integer Evaluation Division	Verify that numeric constants are initially assumed to be integers during calculations.	10/3	3.33333
TC22	Division by Zero (Calculated)	Verify system detects division by zero when it is the result of a nested parenthetical expression.	5/(2-2)	"Division by zero" error
TC23	Inner Parenthesis Evaluation	Verify the system overrides default precedence by evaluating inner parentheses first.	8-(5-2)	5
TC24	Invalid Operator Sequence	Test error handling for unsupported combinations of division and modulo.	5%/2	Error message
TC25	Subtraction Followed by Unary Negative	Test system encountering subtraction operator immediately followed by unary negative operator.	10--2	12
TC26	Addition Followed by Unary Positive	Test system encountering addition operator immediately followed by unary positive operator.	10++2	12
TC27	Testing Infinity Cutoff	Testing immediately under the infinity cutoff	1.8*10**307	1.8e+307
TC28	Testing Infinity Cutoff	Testing immediately above the infinity cutoff	1.8*10**308	inf

3. Environmental needs

3.1.1 Other

Software must be built using a C++ 20 standard compiler via CMake. Text must be encoded using the UTF-8 standard.